EDA Design Flow Integration

Lecture 8: FPGA Flow Practical Evaluation using PYNQ-Z2

辛艾曼 Aman Sinha

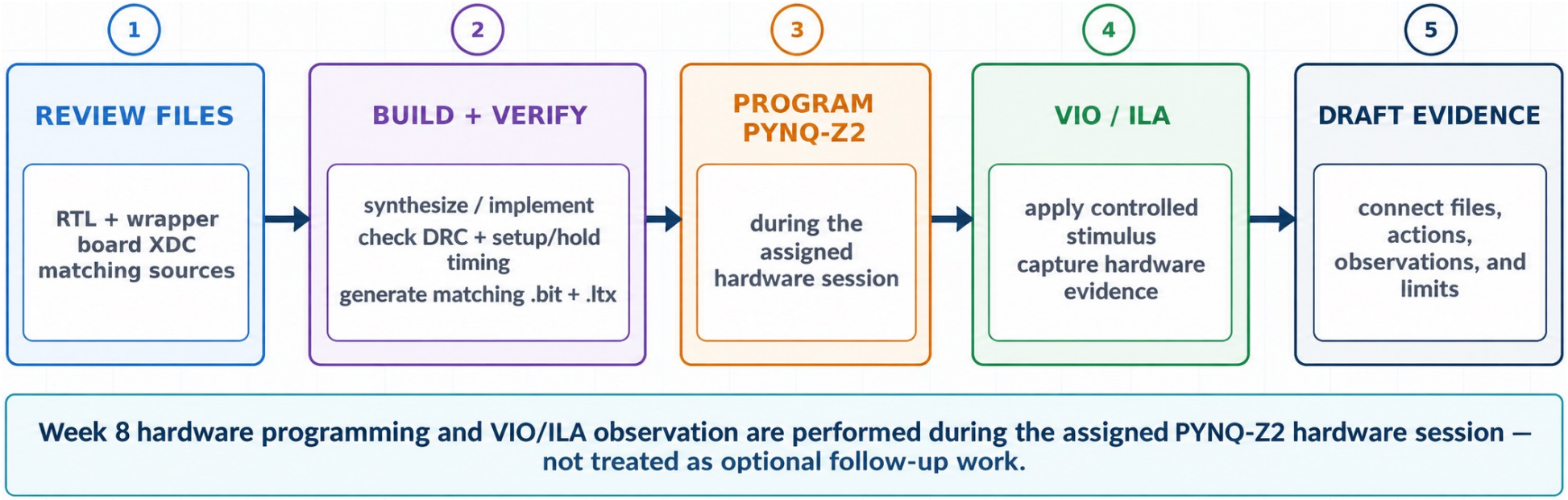
Institute of Integrated Circuit Design

National Sun Yat-Sen University

國立中山大學
National Sun Yat-sen University

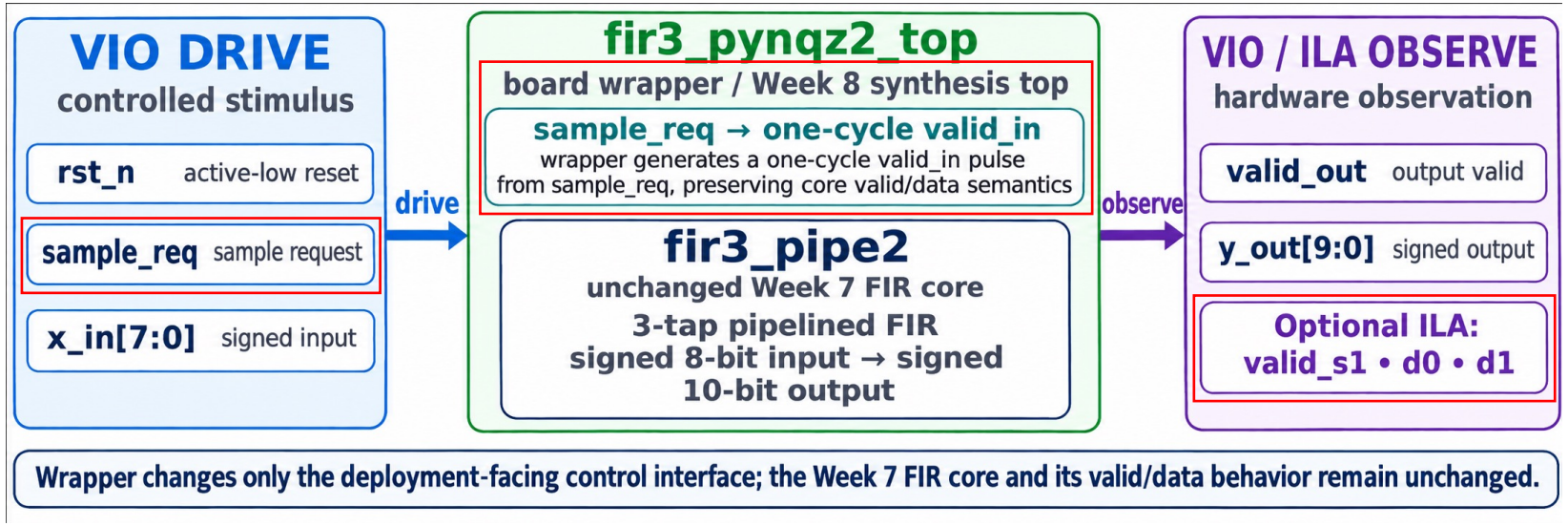
In-Class Work Plan

- Review the Week 8 deployment files and the evidence boundary
- Generate or inspect the bitstream and the programming flow
- Program the PYNQ-Z2 during the assigned hardware session
- Interact with VIO/ILA and capture the required evidence
- Start drafting the evidence-based explanation
- Final 20 min: Matrix-project checkpoint with TA/instructor — verify the previous milestone and record the next action



Guided Deployment Design

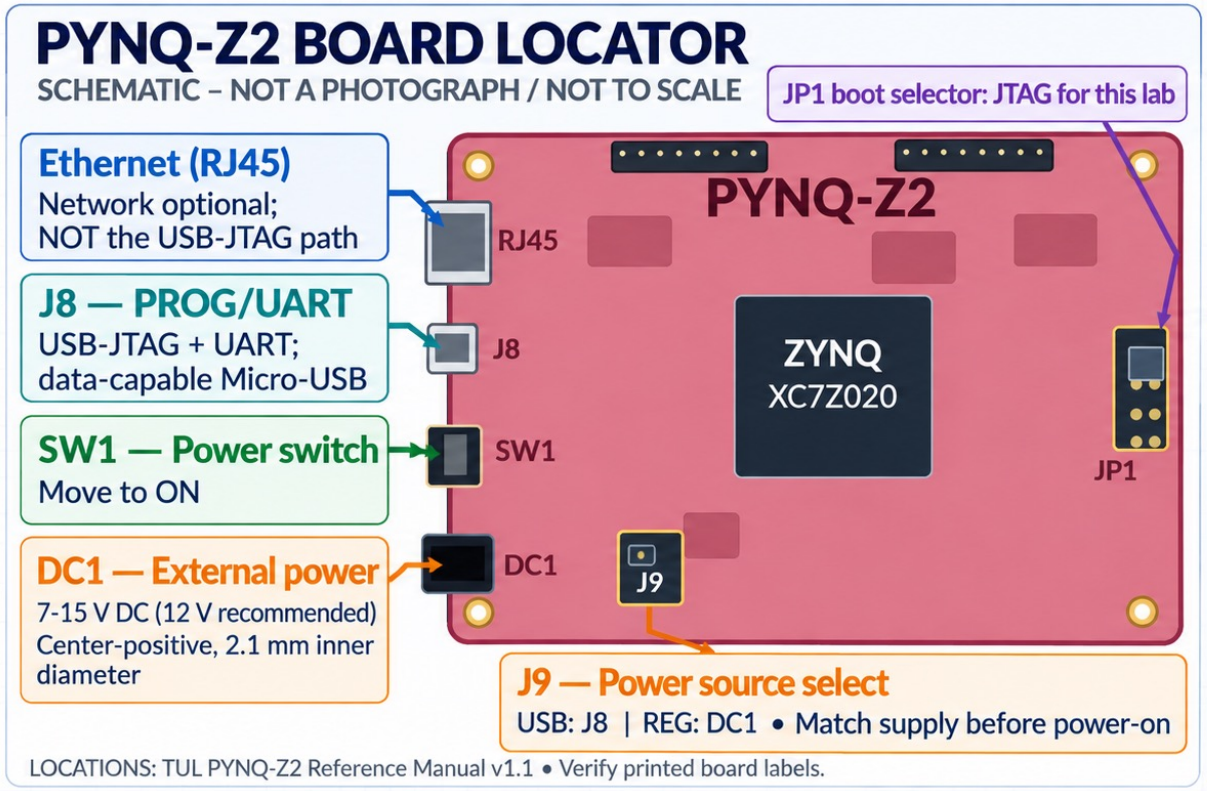
- Use the guided Week 7 FIR design as the core datapath
- Add only the board-facing logic needed for PYNQ-Z2 interaction
- Keep functional behavior consistent with the earlier simulation results
- Use a simple valid/data style interface for controlled testing
- The goal is deployment and observation, not redesigning the algorithm

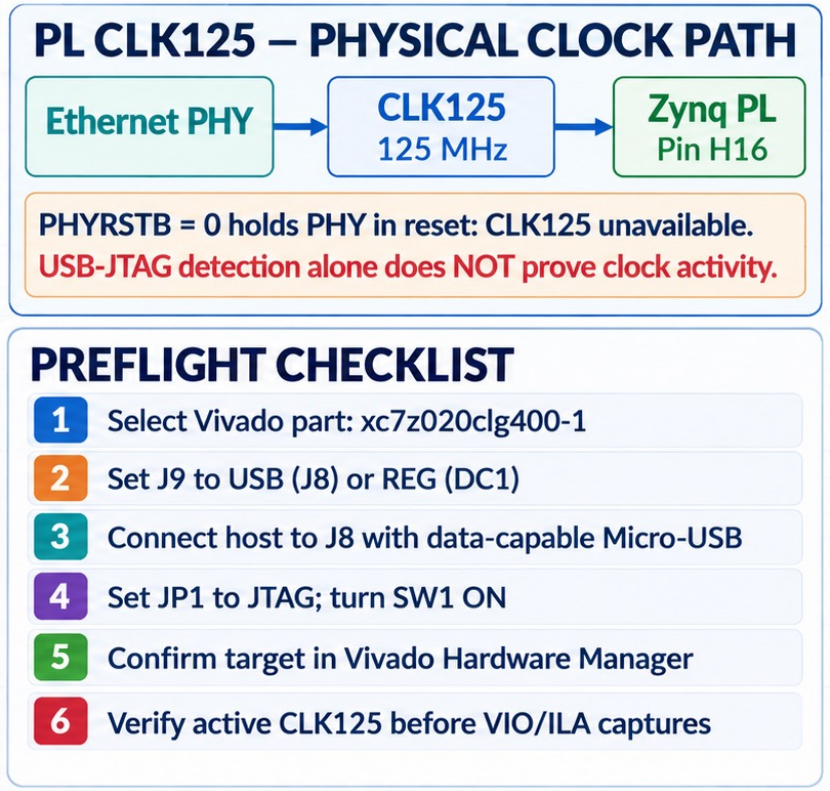


Board Session: Preflight Checklist

1. Select xc7z020clg400-1 as the Vivado target device.
2. Power the PYNQ-Z2 using the correct USB/REG power-jumper setting.
3. Connect a data-capable cable to the PROG/UART Micro-USB port.
4. Set the boot-mode jumper (JP1) to JTAG for this guided session.
5. Verify Hardware Manager discovers the target before programming.
6. Check that the PL reference clock is active; PHY reset can disable CLK125.

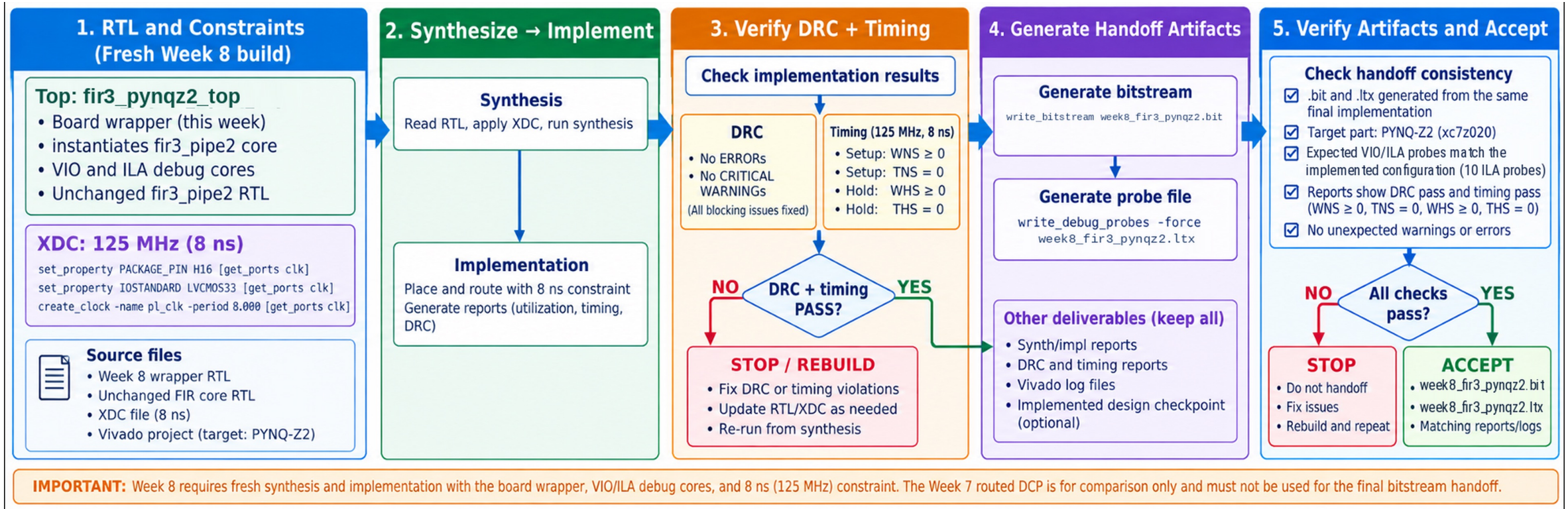
Ethernet is not the JTAG path. For the H16-clocked lab, connect to an active Ethernet port and verify stable CLK125.





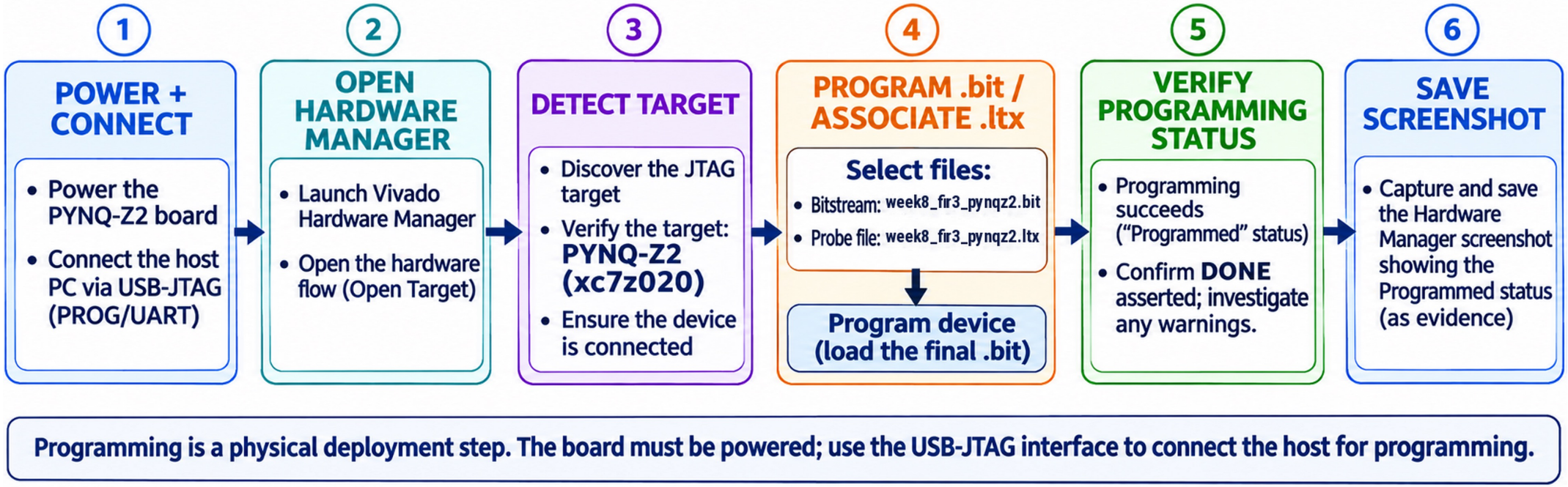
Bitstream Handoff: Accept or Stop

1. Open the final Week 8 board-ready implemented design.
2. Confirm target part, board XDC, DRC, and setup/hold timing status.
3. Generate the .bit file from this final implementation; retain the run log.
4. Export the matching .ltx debug-probes file for VIO/ILA interaction.
5. Pair .bit and .ltx from the same implementation and source revision.
6. Stop if probes are missing or if the board reports do not match the build.



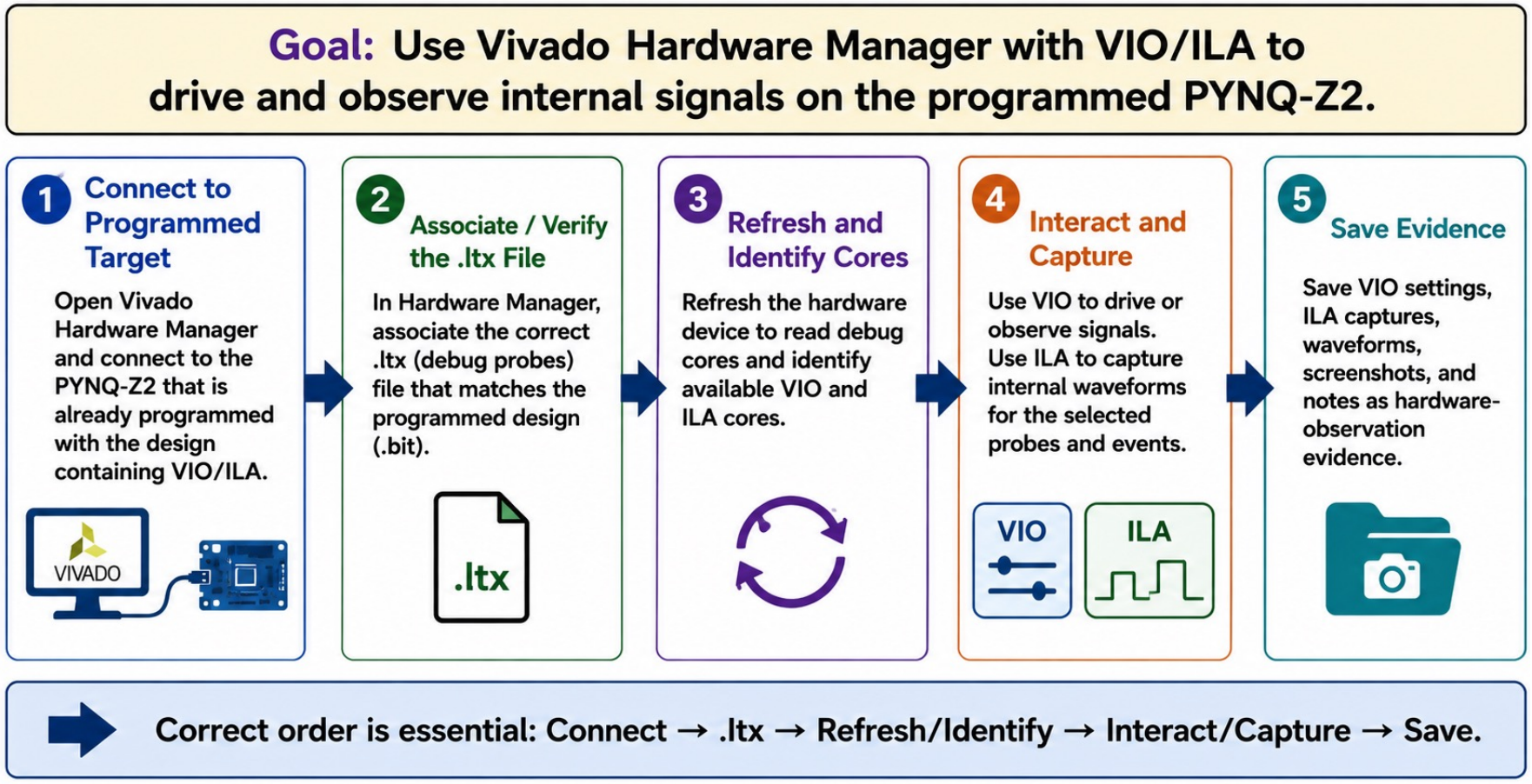
Programming the PYNQ-Z2

- Connect the board and open Vivado Hardware Manager
- Detect the target device through the JTAG connection `ROM and PCIe (faster) are alternatives`
- Program the device with the generated .bit file
- Confirm successful programming before any interactive observation
- Save a screenshot or log entry as deployment evidence



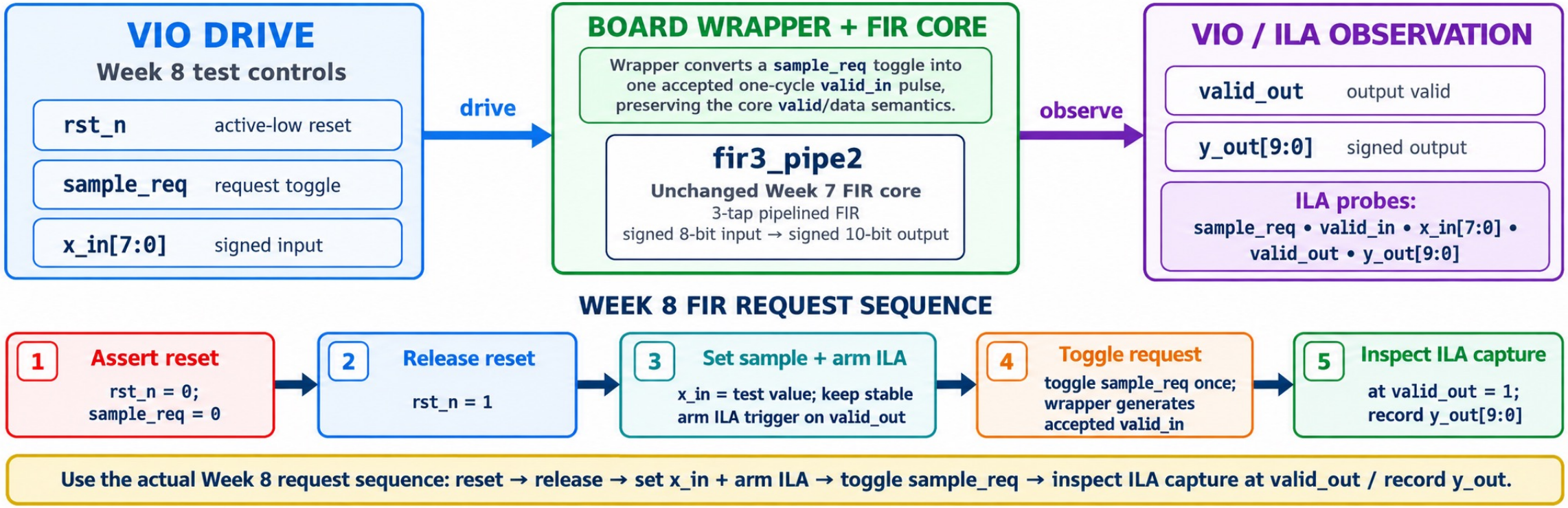
Hardware Observation Goal

- Show that the programmed design responds as expected on hardware
- Use VIO/ILA-style evidence to observe reset, valid, input, and output behavior
- Compare the observed hardware behavior with the expected functional behavior
- Keep the test scenario simple and reproducible
- Hardware evidence complements the matching Week 8 board-ready reports; Week 7 reports remain a baseline



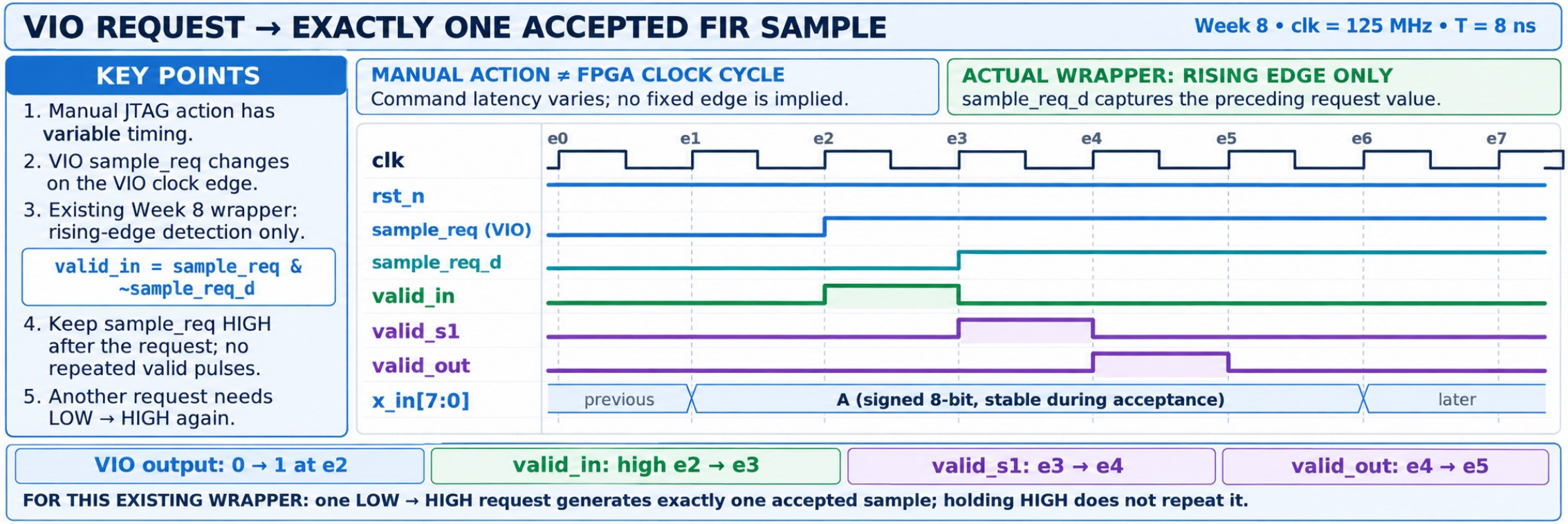
VIO Interaction Basics

- Use VIO to toggle reset and apply simple input or control values
- Change signals in a controlled sequence rather than randomly
- Wait for the design to respond before changing the next stimulus
- Record the key signal values used during the observation
- A clear stimulus sequence improves reproducibility and explanation quality



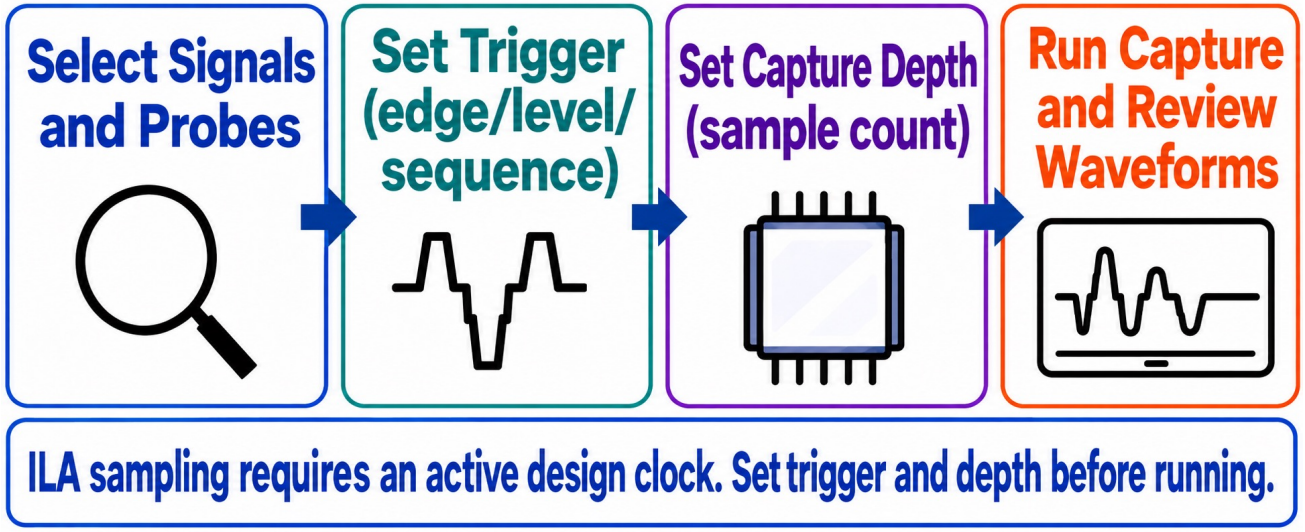
VIO Request: One Transaction Only

- Remember that a VIO output holds its value until it is changed.
- Hold sample_req HIGH after the accepted rising edge; another request requires returning it LOW and then HIGH again.
- Use a VIO request toggle plus synchronized edge detection in the wrapper.
- Hold x_in stable before and throughout the requested transaction.
- Sequence reset low, release reset, set x_in, then toggle the request.
- Verify one request produces one valid output at the expected RTL latency.



ILA Capture Basics

- Select the important internal or top-level signals to probe
- Set an appropriate trigger condition for the event of interest `sample_req`
- Capture only the relevant hardware event window
- Read the capture with the same discipline used for simulation waveforms
- Save a clear screenshot of the final ILA observation



Important: Clock the ILA using the same 125 MHz (`clk`) domain as the monitored FIR signals.
Configure the trigger on `valid_out` and capture `y_out[9:0]` when `valid_out = 1`.

Five Directed FIR Hardware Checks

- **Case 1:** Assert reset; check valid_out follows the RTL reset contract.
- **Case 2:** Send one positive signed input; compare with simulation.
- **Case 3:** Send one negative signed input; inspect the signed output.
- **Case 4:** Send two separated requests; confirm result order and latency.
- **Case 5:** After pipeline drain, change x_in with no new request.
- Record expected and actual valid-qualified outputs for every case.

BEFORE EACH INDEPENDENT CASE: reset the FIR; set sample_req LOW.
For each accepted sample: set x_in, then drive sample_req LOW → HIGH; return LOW to re-arm without accepting a sample.

#	TEST / PURPOSE	VIO INPUT / REQUEST signed x_in[7:0]	EXPECTED OUTPUT signed y_out[9:0]	EXPECTED VALID TIMING relative to accepted request	STUDENT RECORD do not pre-mark
1	RESET Reset contract	rst_n = 0; sample_req = 0 then release rst_n = 1	y_out = 0 on reset valid_out = 0	No valid_out pulse until a new request	Observed: _____ ☐ PASS ☐ FAIL
2	POSITIVE SAMPLE One +5 after reset	One request with x_in = +5 (previous history = 0)	+5	One valid_out pulse at E + 2	Observed: _____ ☐ PASS ☐ FAIL
3	SIGNED BOUNDARY Three −128 samples	−128, −128, −128 3 requests; LOW re-arm between	−128, −384, −512	3 pulses in order each at its own E_i + 2	Observed: _____ ☐ PASS ☐ FAIL
4	SEPARATED REQUESTS Two samples with bubbles	+2, then +3 2 requests separated by idle	+2, +7	2 pulses in order each at its own E_i + 2	Observed: _____ ☐ PASS ☐ FAIL
5	INVALID / BUBBLE No request after drain	Accept +5; wait for drain change x_in to −128; no toggle	Holds last value (+5) no new output	After drain: valid_out = 0 no additional valid pulse	Observed: _____ ☐ PASS ☐ FAIL

TIMING KEY E = edge at which the VIO output sample_req rises
valid_in: E→E+1 | valid_s1: E+1→E+2 | valid_out: E+2→E+3

PASS RULE Compare signed y_out only when valid_out = 1.
No-request test: valid_out stays 0 after drain; output holds.

ILA: Capture One Complete FIR Event

- Probe sample_req, valid_in, valid_s1, valid_out, x_in[7:0], and y_out[9:0]. Confirm that valid_s1 is included in the implemented ILA probes. Rebuild and regenerate matching .bit and .ltx files only if the debug-probe configuration changes.
- Clock ILA from the same active design clock used by the FIR.
- Trigger on sample_req or on a new valid_out event.
- Choose capture depth and trigger position to include the full latency.
- Arm the capture before changing the VIO request control.
- Compare cycle order and valid-qualified y_out with testbench results.

TEST INPUT
After reset: x_in = +5; FIR history = 0

SAMPLED TIMING — NOT A MEASURED CAPTURE
E = S3: first ILA sample observing valid_in = 1

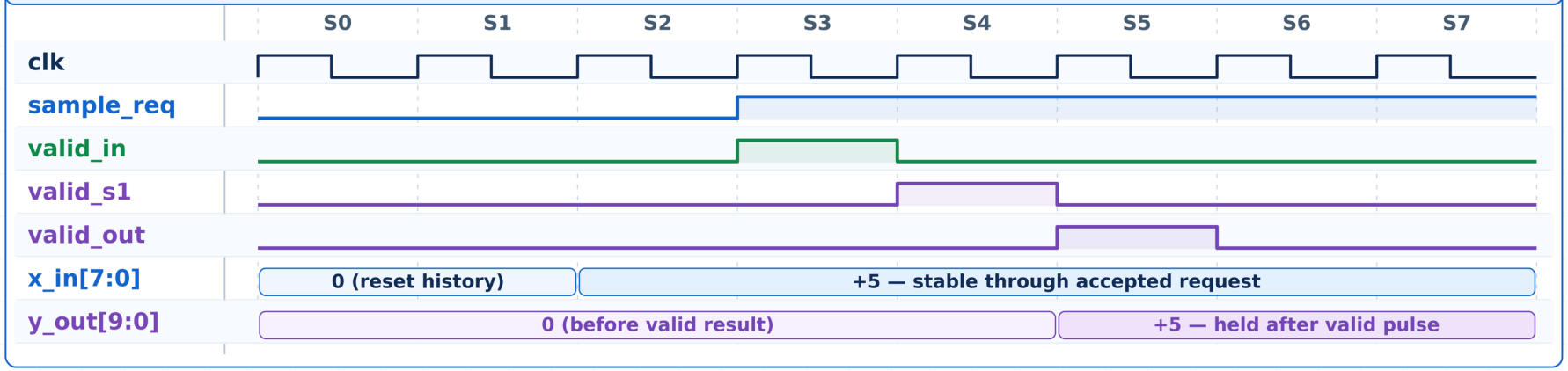
ILA CLOCK AND TRIGGER
Clock = FIR clk | trigger on valid_out rising | pretrigger ≥ 4 samples

CAPTURE THESE SIX PROBES

sample_req	VIO output, rising request
valid_in	accepted one-cycle pulse
valid_s1	internal FIR pipeline valid
valid_out	qualified output valid
x_in[7:0]	signed input (+5)
y_out[9:0]	signed result (+5)

Important: valid_s1 is probe #6.
Add probe; rebuild matching .bit and .ltx.

EXPECTED ILA SAMPLED WAVEFORM • ONE +5 REQUEST



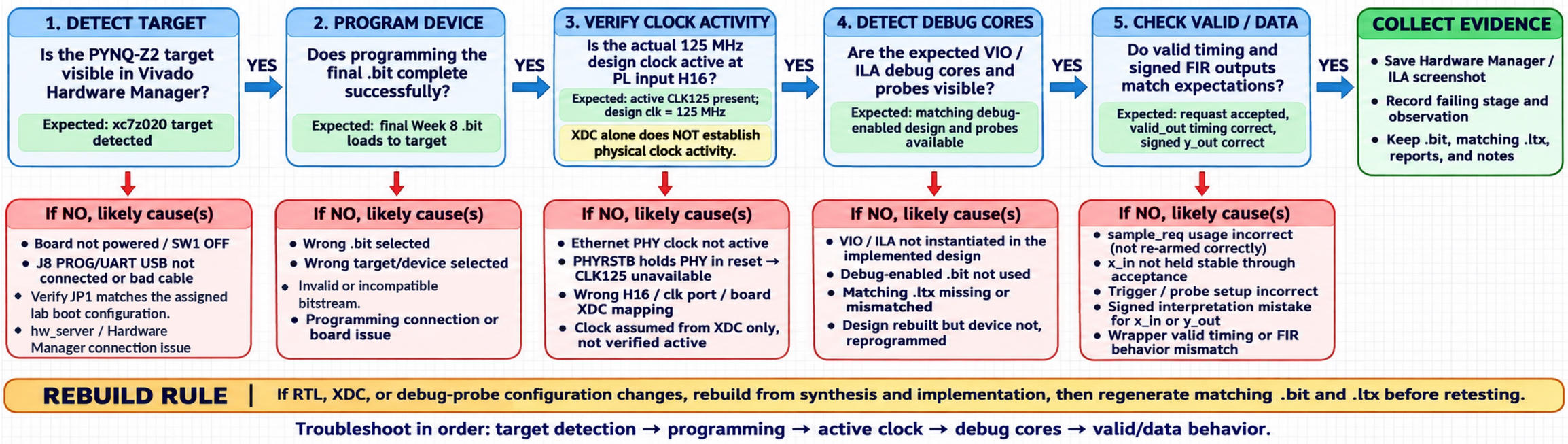
REQUEST → PIPELINE → OUTPUT
S3: valid_in → S4: valid_s1 → S5: valid_out

PASS CHECK
At S5: valid_out = 1 and signed y_out = +5

CAPTURE RULE
S3 is sampled request detection; not VIO click time.

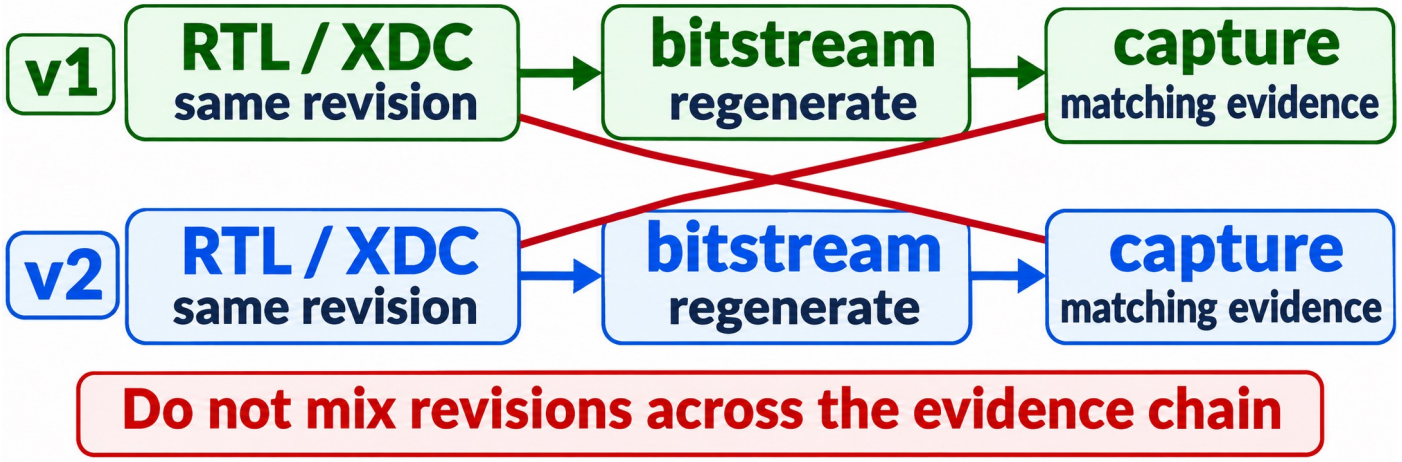
Troubleshoot at the First Failed Stage

- **No target?** Check power, PROG/UART cable, JTAG mode, and hw_server.
- **Cannot program?** Check FPGA part, final .bit, and implementation DRC.
- **No VIO/ILA?** Check debug-core inclusion and matching .ltx probes.
- **No clock activity?** Check H16 clock, PHY reset, ILA clock, and reset.
- **Wrong output?** Check request pulse, signed inputs, latency, and version.
- **Change** one factor at a time; regenerate artifacts after design edits.



Common Mistake: Mixing File Versions

- Do not generate a bitstream from one revision and explain another
- If sources or constraints change, regenerate the bitstream
- Keep the final RTL, XDC, matching .bit and .ltx files, and hardware captures together—all corresponding to the same Week 8 implementation.
- Use a clean folder for the final run
- Document the exact version used for submission



Evidence Record: One Traceable Case

- **Identify** source revision, board XDC, target device, and build ID.
- **Archive** final .bit/.ltx plus matching timing and DRC reports.
- **Record** reset, signed input, request sequence, and expected result.
- **Save** a successful programming screenshot and relevant ILA capture.
- **Compare** observed valid_out/y_out against the saved simulation oracle.
- **Claim** only that the tested case passed or failed on this programmed board.

1. BUILD IDENTITY

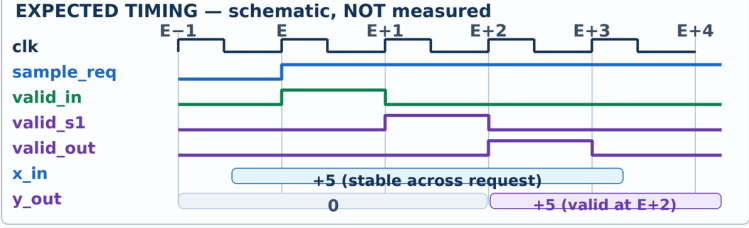
Source revision	[exact Git SHA / package hash]
Build / run ID	[record exact Vivado build ID]
Synthesis top	fir3_pynqz2_top
RTL source files	rtl/fir3_pipe2.v rtl/fir3_debug_wrapper.v rtl/fir3_pynqz2_top.v
Board XDC	constraints/fir3_pynqz2.xdc
Part + clock	xc7z020clg400-1 H16 / CLK125 / 8.000 ns
Bitstream (.bit)	[record exact generated .bit path]
Debug file (.ltx)	[record exact matching .ltx path]
Implementation reports	[record timing report] [record DRC report]

Use the same implementation build for .bit, .ltx, timing/DRC reports, and board captures.

2. DIRECTED TEST

Test / purpose	Case 2 — one positive sample (+5)
Reset / setup	rst_n = 0, sample_req = 0 Release rst_n = 1; history = 0
VIO stimulus	Set signed x_in[7:0] = +5
Accepted request	sample_req: LOW → HIGH at E Hold x_in stable through acceptance
Expected timing	valid_out: E+2 → E+3 (one cycle)
Expected result	signed y_out[9:0] = +5 at E+2

EXPECTED TIMING — schematic, NOT measured



3. HARDWARE EVIDENCE

ILLUSTRATIVE TEMPLATE — NOT MEASURED

Programming	DONE / device status: [record]
Run / target	[record actual board and build ID]
Observed valid_out	[enter pulse cycle and duration]
Observed signed y_out	[enter measured signed value]
ILA capture (.ila)	[record exported .ila filename]
Screenshot (.png)	[record Hardware Manager .png]
Oracle comparison	Expected: E+2, signed +5

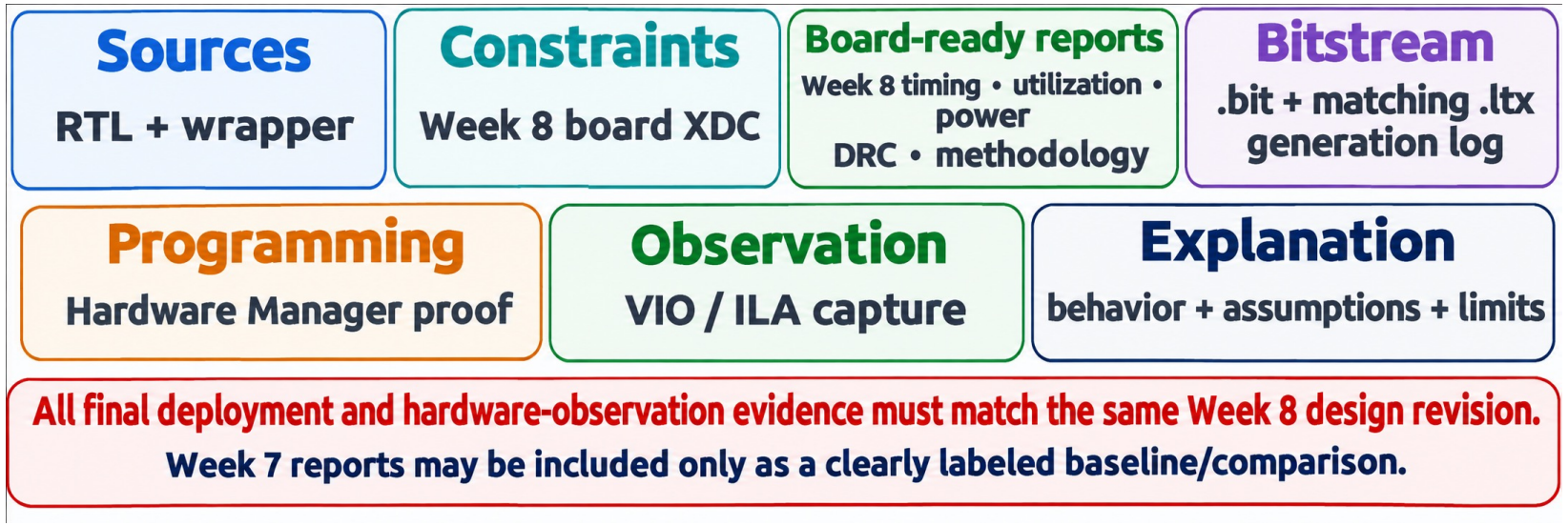
VERDICT: PENDING HARDWARE TEST
[ ] PASS [ ] FAIL
Mark only after matching the saved oracle.

TRACEABILITY RULE

Record exact revision, build ID, RTL, XDC, matching .bit/.ltx, timing/DRC, stimulus, expected result, measured result and raw .ila capture together.
Do not claim PASS from a schematic or simulation alone.

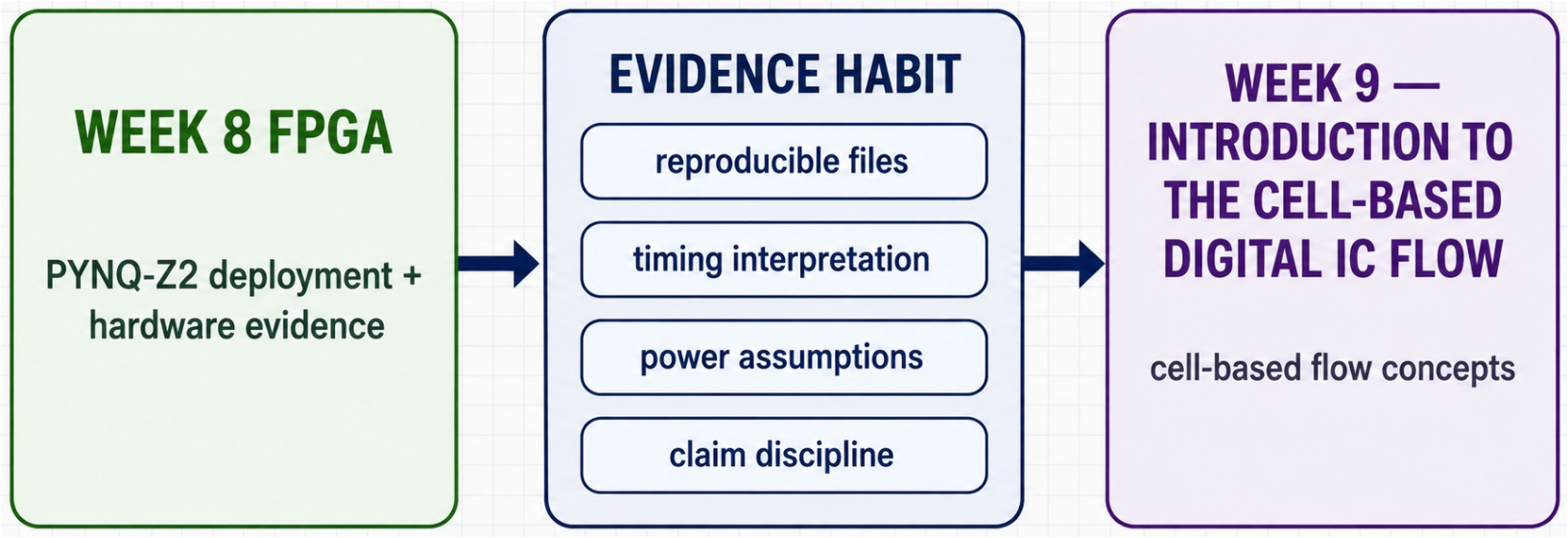
Submission Checklist

- Core RTL/wrapper and constraint files used for deployment
- Bitstream-generation evidence and a programming screenshot or log
- Week 8 board-ready timing, utilization, power, DRC, and methodology evidence; Week 7 reports only as a labeled baseline/comparison
- VIO/ILA or Hardware Manager observation evidence
- Short written explanation of behavior, assumptions, and limits



How Week 8 Prepares Week 9

- Week 8 completes the guided FPGA path through deployment and hardware observation
- Week 9 introduces the cell-based digital IC flow
- The same evidence discipline will carry over to the new tool flow
- Later comparisons will separate FPGA-specific resources from IC metrics
- Implementation assumptions and evidence quality remain central



Exit Reflection

- What evidence proves bitstream generation
- What evidence proves the board was programmed
- What evidence shows the hardware behavior
- What claim still should not be made from this evidence
- How do Week 7 and Week 8 evidence complement each other

